G1 Campaign 2 — Deployment & Test Protocol

Adrián Lendínez Ibáñez — University of Bedfordshire, 21 July 2026. Complete runbook to deploy the upgraded stack and collect the campaign-2 dataset on the real G1. Campaign 1 is closed; its data remains the "before" corpus. New since campaign 1 (all merged to main): governed multi-parameter profiles + turn governance, door-approach analogy library with twin-explored trust, graded spill→speed coupling, platform-calibrated fragility (REALCAL), sim-to-real trust transfer, camera brake for below-laser furniture, sticky door-frame cells, GPU vision (metric depth + SegFormer + YOLO), shadow particle filter, covariance-grade logging.

0. One-time deployment (GPUEDGE, ~20 min)

```
cd ~/Documents/G1_UNITREE_ROBOT_META_REASONING
git pull                                    # brings all campaign-2 code


# GPU vision (the 2 NVIDIA GPUs; heavy install, once):
source ~/g1env/bin/activate
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
pip install -r requirements-perception.txt
```

Verify vision (no robot needed). Launch the perception server, then check health and run a smoke inference on a logged frame:

```
G1_FLOORCOLOR=1 python perception_server.py --host 0.0.0.0 --port 8008 \
  --fx 300 --fy 300 --cx 160 --cy 90 --cam-h 1.10 --cam-pitch -10   # frames are 16:9 (320x180)!
# (defaults already enable depth on cuda:0 and seg/det on cuda:1)


curl -s http://127.0.0.1:8008/health        # expect models {depth, seg, det} + 2 GPUs


python3 - <<'EOF'
import base64, json, urllib.request
img=''+base64.b64encode(open('dataset/20260714_171700_ours_B_t000s.jpg','rb').read
()).decode()
r=urllib.request.urlopen(urllib.request.Request('http://127.0.0.1:8008/perceive',
  json.dumps({'image':img,'pose':[0,0,0],'hband':[0.10,1.30],'max_range':3.0}).encode(),
  {'Content-Type':'application/json'}))
d=json.load(r); print('scan pts:',len(d.get('scan',[])),'dets:',[x['label'] for x in
d.get('detections',[])])
EOF
```

PASS = scan points > 0 and detections listed. If the depth model fails to load, the run still works but `perc_n` stays near 0 — vision then only warns via the colour channel.

Optional (recommended once vision passes): design resolution. Capture at 640 px and use the server's native intrinsics:

```
# g1_goto side (add to every run env):   G1_CAM_W=640 G1_CAM_Q=0.7
# server side (replace the intrinsics):   --fx 600 --fy 600 --cx 320 --cy 180
# RULE: capture width and intrinsics must always match (320x180-> fx300 cx160 cy90; 640x360-> fx600 cx320
cy180).
```

1. Session start ritual (EVERY robot boot)

The vendor app update made the robot's Wi-Fi password **rotate on every boot** — never join from iPhone Settings.

- Boot the robot; wait for full start-up.
- iPhone: Bluetooth ON → connect **from the app** (first time after update: Settings → Wi-Fi → Forget the robot network). Verify teleop works in-app.
- USB to GPUEDGE → `ios_webkit_debug_proxy` → verify it lists the app WebView.

- `python g1_teleop.py` → stick taps move the robot (confirms injection survived any app change).
- Terminals: perception server (§0 command) · `python spill_mark.py` (leave running all session; ENTER per spill; a burst of slosh = ONE mark) · main terminal for runs.
- `python g1_goto.py noisecheck` — 20 s standing: the day's sensor-noise floor (saves dataset/<ts>_noise.json).
- Walk the robot near a waypoint → `python g1_goto.py reloccheck`. A mid-session robot reboot breaks the phone link (new key) → redo this ritual and note it in the run log.

2. Shakedown — 2 DRY runs (no water) before any data run

Verifies on real physics what the twin cannot guarantee: door crossing with the governed body radius, camera brake not interfering, sticky frame cells, and the new log fields.

```
G1_META2=2 G1_M2_CFG=config_meta2_g1payload.json G1_M2_REALCAL=1 \
G1_M2_STATE=meta2_state_c2_shakedown.json G1_M2_STATE_INIT=meta2_state_twin_explored.json \
G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_DOORLIB=1 G1_M2_FRAGSPEED=1 \
G1_PERC=127.0.0.1:8008 python3 g1_goto.py gotoviz B      # then: gotoviz A
```

PASS checklist: reaches both ways · door crossed without contact · `perc_n` in the DOZENS in the printed line (vision live) · no unexpected COLOR-BRAKE stops in open space · after the run, the dataset JSON has `meta2_pf` and `meta2_unc` in samples. If the robot stops in front of the sofa corridor briefly and creeps — that is the camera brake working as designed.

3. Optional but recommended: furniture survey (10 min, once)

Registers the sofa arm and the low cabinet in the static map layer so the planner routes around them proactively (the camera brake stays as safety net). With the robot standing 1–1.5 m in front of each item, run the NEW command G1_PERC=127.0.0.1:8008 python3 g1_goto.py furniture 15 (accumulates VISION depth cells — the old waypoint sweep is laser-based and cannot see low furniture). Repeat from a second angle; it prints each added cell — verify they match the real item. REQUIRED before water runs: in run 153921 the camera brake correctly prevented contact at the unmapped sofa, but the planner then stalled through it → governed HELP at 22.8 s; with the sofa in the map, A* routes around and the brake stays a safety net. Cells are stored in `nav_map.json` near (−1.1, 0.4) and (1.0, 0.5).

4. Campaign-2 data runs

Water protocol (unchanged from campaign 1): weigh the cup BEFORE and AFTER every leg; never refill between A→B and B→A of the same pair (mixes fresh-cup and settled-cup physics); refill to target only when starting a new pair; note grams + marks per leg. Spill marker: one ENTER per slosh burst. If the marker dies mid-run: Ctrl-C the run, relaunch marker, mark the run INVALID, repeat it. A governed HELP stop is a VALID result. Never edit code while a run is in flight.

Arm	Runs	Fill	Command core	What it yields
C2-GOV (headline)	6 (3 pairs, alternate B/A)	200 g	governed full-stack (below)	governed completion times, spills/mark economics, door behaviour with library, PF posteriors
C2-BASE	4 (2 pairs)	200 g	`G1_META2=0 G1_PERC=... gotoviz B`	baseline comparison at same fill/same code platform
C2-FULLCUP	2 governed + 2 baseline	247 g	same commands	THE headline pair: campaign-1 governed aborted 4/4 at full cup (ungoverned turns); turn cap should now survive it
C2-WRONG (R3)	6	200 g	governed but `G1_M2_CFG=config_meta2_g1door.json` + fresh `G1_M2_STATE=meta2_state_c2_wrong.json`	the DST recovery figure: write down Pl after EVERY run

			(rm once) — keep REALCAL	
C2-LID (R4)	4 (2 base + 2 governed)	200 g	sealed lid; governed uses door cfg + `G1_M2_STATE=meta2_state_c2_lid.json`	lid condition data + validates the particle filter: `meta2_pf` P(lid open) should fall toward 0
C2-ABL (optional)	2	200 g	governed + `G1_COLORBRAKE=0 G1_DOORSTICKY=0`	ablation of the two capability fixes if RQ wants it isolated

Governed full-stack command (C2-GOV / C2-FULLCUP):

```
G1_META2=2 G1_M2_CFG=config_meta2_g1payload.json G1_M2_REALCAL=1 \
G1_M2_STATE=meta2_state_c2_m2.json G1_M2_STATE_INIT=meta2_state_twin_explored.json \
G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_DOORLIB=1 G1_M2_FRAGSPEED=1 \
G1_PERC=127.0.0.1:8008 python3 g1_goto.py gotoviz B     # alternate with: gotoviz A
```

Keep `meta2_state_c2_m2.json` across all C2-GOV runs (trust accumulates — that IS the system). After each governed run, note the state:

```
python3 -c "import json;print(json.load(open('meta2_state_c2_m2.json')))"
```

If Pl(Cautious_Nav) ever drops below 0.45, STOP and report — deploy-veto inversion would be back (should not happen under REALCAL relative attribution).

5. After every block: commit the data

```
git add dataset/ runs_summary.csv meta2_state_c2_*.json goto.log
git commit -m "C2 <arm>: <n> runs, fills <g>, notes"
git push
```

Per run you get: `dataset/<ts>_ours_<goal>.json` (samples with `meta2_pf`/`meta2_unc`/`meta2_rho`, events, laser snapshots with pose+motion), film JPGs, and the updated state files. Report weights + marks in chat as in session 1 — the accounting and the interim analysis are produced from exactly these.

6. What each arm feeds (paper/thesis mapping)

- C2-GOV vs C2-BASE at 200 g → the governed-vs-baseline table (times, marks, g/mark, collisions) with all fixes active.
- C2-FULLCUP → the headline claim: governance turns a 4/4-abort condition into completions (turn cap = the campaign-1 root cause removed).
- C2-WRONG Pl sequence → DST recovery under wrong prior on hardware (thesis §5.3.6.3 analogue) with platform-relative attribution.
- C2-LID → covered-delivery condition + particle-filter validation on real sealed-lid data (P(open)→0) → enables the v2 belief-conditioned QoE hook.
- All arms → covariance corpus (laser snapshots with motion phase), Σ(r,θ) estimation offline; `meta2_unc`/`meta2_pf` → the POMDP framework note's estimator comparison (DST vs PF) on real data.

7. Troubleshooting quick reference

Symptom	Cause	Fix
iPhone: "incorrect password" joining robot Wi-Fi	Rotating key (app update)	Forget network; connect FROM the app via Bluetooth (§1)
Proxy lists no WebView	App not front / USB	Reopen app, replug USB, restart proxy
`perc_n` ≈ 0 during runs	Depth model not loaded	`curl :8008/health`; reinstall §0; run still valid but vision-

		degraded — note it
Unexpected stop in open space, phase tag `!C`	Camera brake false positive	Note position; `G1_COLORBRAKE=0` for the next run and report
Robot loops / HELP at door or pocket	Valid governed outcome	Let P9 finish; log it; weigh the cup; it is data
Reloc jumps after a bump	Localization lost	Stop session runs; walk to waypoint; `reloccheck`; mark run INVALID if mid-run
Marker died mid-run	Instrumentation gap	Run INVALID: Ctrl-C, relaunch marker, redo (§4)

Everything here is reversible: `G1_COLORBRAKE=0`, `G1_DOORSTICKY=0`, omit the profile/library flags, or `G1_META2=0` returns the platform to campaign-1 behaviour for any control run RQ requests.